

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 06

ভার্চুয়াল মেমোরি

নমস্কার। এই ভিডিওতে, আমরা দেখবো ভার্চুয়াল মেমরি; যা আজকের দিনে সিস্টেমে সবচেয়ে বেশি ব্যবহৃত মেমরি ম্যানেজমেন্ট টেকনিক।

(স্লাইডটাইপ পড়ুন: 00:31)

সুতরাং একটি ভার্চুয়াল মেমোরি সিস্টেমের মধ্যে, পুরো RAM যা সিস্টেমে উপস্থিত থাকে সেটি পেজ ফ্রেম নামে পরিচিত সমান আকারের পার্টিশনে বিভক্ত হয়। তাই সাধারণত, পেজ ফ্রেম 4 কিলোবাইট প্রতিটি হবে। সুতরাং এই RAM এর মধ্যে, উদাহরণস্বরূপ, আমাদের 14টি পেজ ফ্রেম আছে; যা 1 থেকে 14 সংখ্যায়ুক্ত। এবং, এই পেজের প্রতিটি ফ্রেমের একই আকার আছে। পূর্ববর্তী ইন্টেল প্রসেসরের মধ্যে সমস্ত পেজগুলি 4 কিলোবাইট এ নির্দিষ্ট করা হয়েছিল। কিন্তু, সাম্প্রতিক প্রসেসরগুলিতে আমাদের পেজ বা পেজ ফ্রেম থাকতে পারে, যা আগের চেয়ে বড় আকারের। অনুরূপভাবে, প্রসেস যা সিপিইউ তে এক্সিকিউট হয় বা প্রসেসের সাথে সংশ্লিষ্ট প্রসেস ম্যাপে সংশ্লিষ্ট করে তা সমান আকারের ব্লকগুলিতে বিভক্ত হয়। এখন, একটি প্রসেসকে এমনভাবে ভাগ করা হয় যে এই প্রসেসের ব্লক পেজ ফ্রেমের আকারের সমান; এই প্রসেসের প্রতিটি ব্লকের আকার পেজের আকারের সমান। এখন যেহেতু আমরা এই ভাবে র‍্যামটিকে বিভক্ত করেছি, প্রসেসরের মেমরিও একইভাবে বিভক্ত, আমরা কি করতে পারি তা হল প্রসেসের ব্লকগুলি RAM এর পেজের ফ্রেমগুলিতে তে বরাদ্দ করা। উপরন্তু, অপারেটিং সিস্টেম কীভাবে বজায় রাখে তা হল একটি টেবিল। এই টেবিলটি উপস্থিত RAM-র মধ্যে রয়েছে এবং এখানে দেখানো হয় নি। কিন্তু, কি কি রয়েছে তা প্রসেস ব্লক থেকে পেজ ফ্রেমের ম্যাপিং। উদাহরণস্বরূপ, এখানে ম্যাপিং খুব সহজ;

ব্লক 1 প্রসেসটি পেজ ফ্রেম নম্বর 1 এ ম্যাপ করা হয়, অর্থাৎ প্রসেসের ব্লক 1 টি পেজ ফ্রেম নম্বর 1 এ ম্যাপ, ব্লক 2 টি পেজ ফ্রেম নম্বর 2 এ ম্যাপ, ব্লক 3 টি পেজ ফ্রেম নম্বর 3 এ ম্যাপ, এবং আরো অনুলিপি করা হয়। এখন এই, আমি আবার মনে করাতে চাই; এটি একটি প্রসেস প্রতি পেজ টেবিল। সুতরাং প্রসেস প্রতি মানে সিস্টেমে চলমান প্রতিটি প্রসেসের একটি অনুরূপ পেজ টেবিল থাকবে যেমন এখানে দেখানো হয়েছে।

(স্লাইডটাইপ পড়ুন: 03:23)

এখন আমাদের এমন একটি টেবিল আছে যা সংশ্লিষ্ট পেজ ফ্রেমের সাথে একটি প্রসেসের ব্লকগুলির মধ্যে ম্যাপিং করে, তাহলে আমরা কি করতে পারি আমরা যে কোনও ধরনের ম্যাপিং নির্বাচন করতে পারি। উদাহরণস্বরূপ, এখন ব্লক 14-এ যে প্রসেসটির ব্লক 1 আছে সেটি হচ্ছে প্রসেসের ব্লক 2 পেজ ফ্রেম 2 তে আছে, পেজ 13-এ প্রসেসের ব্লক 3 এবং আরও অনেক কিছু। মূলত, আমরা এই নির্দিষ্ট প্রসেসের পেজ টেবিল কীভাবে অর্জন করতে পারি তা হল এই প্রসেসটির ব্লকগুলি র‍্যামের সংলগ্ন পেজের ফ্রেমগুলিতে থাকবে এমন নয়। যাইহোক, অতিরিক্ত সমস্যাটি হচ্ছে প্রতিবার একটি নির্দিষ্ট মেমরি অংশের প্রসেসরের মেমরির ম্যাপ অ্যাক্সেস প্রয়োজন। CPU- কে প্রসেসরের পেজ টেবিলের সন্ধান করতে হবে, সেই নির্দিষ্ট ঠিকানার জন্য সংশ্লিষ্ট পেজ ফ্রেম নম্বরটি প্রাপ্ত করতে হবে এবং শুধুমাত্র তখনই র‍্যামের ঠিকানা অ্যাক্সেস করা যাবে। উদাহরণস্বরূপ, আসুন আমরা বলি যে CPU- এ এক্সিকিউট করা প্রোগ্রামটি একটি মেমরি অ্যাক্সেস করে যেটা প্রসেসের ব্লক 3 এর ভিতরে অবস্থান করে। সুতরাং, এটি একটি নির্দেশ বা একটি লোড বা কিছু তথ্য সংরক্ষণ করতে পারে। এখন, যখন এই ধরনের অপারেশন প্রসেসরে উপস্থিত একটি মেমরি ম্যানেজমেন্ট ইউনিট কে এক্সিকিউট করে তখন এই অ্যাক্সেস আটকাচ্ছে। এবং তারপর, এটি পেজ টেবিলের দিকে তাকিয়ে দেখবে আর, সংশ্লিষ্ট পেজ ফ্রেমটি যেটি ব্লক 3 ছিল সেটি হল 13। তারপর এটি একটি ফিজিক্যাল অড্রেস হিসাবে পরিচিত কিছু তৈরি করতে যাচ্ছে, যা পরে RAM এর 13 তম পেজ ফ্রেমে পাওয়া যাবে। ফলস্বরূপ, প্রতিটি মেমরির অ্যাক্সেসের জন্য পেজ টেবিল অ্যাক্সেস করতে অতিরিক্ত চাপ রয়েছে, যেটি র‍্যামের অ্যাক্সেসের আগেই সম্ভব হতে পারে। তাই এই পেজ টেবিল অ্যাক্সেস করতে অতিরিক্ত ওভারহেড হয়। এবং, সাধারণত এই ওভারহেড আংশিকভাবে একটি TLB ক্যাশ বা একটি ট্রান্সলেসন লুকআপ সাইড(Translational Lookaside) বাফার ক্যাশ হিসাবে পরিচিত কিছু ব্যবহার করে কিছুটা সমাধান হয়। তাই, আমরা TLB ক্যাশ সম্পর্কে বিস্তারিত জানব না। কিন্তু, যে কোর্সের আমরা পড়ছি তার প্রতি আমাদের বোধগম্যতার জন্য, আমাদের মনে রাখতে হবে যে প্রতিটি মেমরি অ্যাক্সেস বা প্রতিটি লোড বা ইনসট্রাকশন ভাঙার যেগুলি প্রসেসটি তার এক্সিকিউশনের সময় আনে প্রসেসটির নির্দিষ্ট পেজ টেবিলের সন্ধান করতে হবে এবং শুধুমাত্র তখনই RAM অ্যাক্সেস করা যাবে।

(স্লাইডটাইপ পড়ুন: 06:২9)

এখন, আমরা কি উল্লেখ করেছি যে সিস্টেমের মধ্যে সঞ্চারিত প্রতিটি প্রসেসের নিজস্ব প্রসেস পেজ টেবিল থাকবে। সুতরাং, যদি আপনার এটি দ্বিতীয় প্রসেস হয় তবে প্রসেস 2, একটি প্রসেস 2 সংশ্লিষ্ট পেজ টেবিলেরও থাকবে; প্রসেস 3 এর সাথে সম্পর্কিত পেজ টেবিলও থাকবে। এখন, এই প্রসেস বা এই প্রসেসগুলি আমরা এই প্রসেসগুলির সাথে যুক্ত মেমরি দেখেছি মেমরির ইউজার স্পেস অঞ্চলে। যাইহোক, প্রসেস পেজ টেবিল কার্নেলে উপস্থিত থাকে। এবং সেইজন্য, কোনও প্রোগ্রাম যা আপনি ইউজার স্পেসে লিখেছেন, কিন্তু প্রসেস পেজ টেবিলের ম্যাপিং কি তা নির্ধারণ করতে সক্ষম হবেন না। এখন, আমাদের সিস্টেমে একাধিক প্রসেস চলছে, প্রতিটি প্রসেসের বিভিন্ন পেজ ফ্রেম ম্যাপিং থাকতে পারে। উদাহরণস্বরূপ, প্রসেস 1 এর এই বিশেষ র‍্যামে সমস্ত নীল পেজ ফ্রেমগুলির সঙ্গে ম্যাপ রয়েছে; যখন প্রসেস 2 এর সব হলুদ পেজ ফ্রেম আছে; তৃতীয়, প্রসেস 3 সব কমলা পেজ ফ্রেম আছে। এখন, আমরা দেখতে পাচ্ছি যে আমরা একসাথে একাধিক প্রসেস সহ RAM ভাগ করতে সক্ষম। এখন, অনুমান করুন যে আমাদের সিস্টেমে একটি সিপিইউ আছে। এর মানে হল যে একটি নির্দিষ্ট সময়ে, শুধুমাত্র একটি প্রসেস চালানো হবে। এখন, এই প্রসেসটি চলতে থাকবে যতক্ষণ পর্যন্ত সময় স্লাইস সম্পূর্ণ না হয়। সুতরাং, প্রসেসের এক্সিকিউট করার সময়, উদাহরণস্বরূপ যখন, প্রসেস 1টি CPU-র মধ্যে এক্সিকিউট করা হয়, এই বিশেষ প্রসেস পেজ টেবিলটি সক্রিয় পৃষ্ঠার টেবিল হবে। অতএব, যে কোনও নির্দেশনা বা তথ্য লোড করা বা সংরক্ষণ করা হবে, এই নির্দিষ্ট প্রসেস পেজ টেবিলের মধ্যে সংশ্লিষ্ট পেজের ফ্রেমটি RAM-র মধ্যে পেতে হবে। প্রসেস 1 থেকে প্রসেস 2 তে যখন কন্টেকস্ট সুইচ হয় এটি এই প্রসেসটির পেজ টেবিলে হবে, যেটা সক্রিয় হবে। সুতরাং, যে কোনও ঠিকানা যা অ্যাক্সেস করা হয় প্রসেস 2 থেকে এই সক্রিয় পেজ ফ্রেমের থেকে সংশ্লিষ্ট পেজের ফ্রেম নম্বরটি পাবে। অনুরূপভাবে, যখন প্রসেস 3 টি CPU-এ এক্সিকিউট হয়, তখন এই প্রসেসটির পেজ টেবিলটি সক্রিয় পেজের টেবিল হবে। তাই, আমরা যা দেখতে পাচ্ছি যে প্রসেস 1 বা প্রসেস -3 এর অনুরূপ কোন পেজ ফ্রেম অ্যাক্সেস করা প্রসেস 2 এর পক্ষে সম্ভব নয়।

(স্লাইডটাইপ পড়ুন: 09:37)

পরের জিনিসটি আমরা দেখব যে আমরা প্রসেসটি চালানো শুরু হওয়ার আগে কি সত্যিই সব ব্লকগুলি মেমোরিতে লোড করতে হবে? সুতরাং, আগের ভিডিওতে আমরা দেখেছি যে যখন একটি প্রসেস চালানো হয়, তখন পুরো প্রসেসটি RAM এ লোড করা হয়েছিল। সুতরাং, এখন আমরা নিজেদেরকে জিজ্ঞাসা করি যে এই ধরনের একটি লোড আসলে প্রয়োজন কী? এবং উত্তর হল না। মূলত, প্রোগ্রামগুলি সম্পর্কে আমরা যা লক্ষ্য করি তা হল এই যে প্রোগ্রামগুলির সমস্ত অংশ একযোগে অ্যাক্সেস হয় না। অন্য কথায়, কোনও নির্দিষ্ট কর্মসূচির কার্যকারিতার ক্ষেত্রে একটি এলাকাভিত্তিক দৃষ্টিভঙ্গি রয়েছে। এর অর্থ এই যে, যদি কিছু ইন্সট্রাকশন এক্সিকিউট করা হয়, ধরি এই নির্দিষ্ট ব্লকটি, এটি সম্ভবত পরবর্তী নির্দেশনা বা পরবর্তী কয়েকটি ইন্সট্রাকশনও এই ব্লকটিতে উপস্থিত আছে। সুতরাং, একটি নির্দেশ আছে ব্লক 3 তে, এটি সাভাবিক যে প্রোগ্রামটি যেহেতু এলাকাভিত্তিক এই ব্লকের বেশ

কয়েকটি ইন্সট্রাকসন এক্সিকিউট হবে সেই সাথে আরও কিছু। আসলে, এটি প্রায়শই ঘটে যে প্রসেস মেমোরির বেশ কয়েকটি অংশ থাকতে পারে, যা এমনকি সবগুলি অ্যাক্সেসও করা যায় না। অর্থাৎ, তারা এক্সিকিউট হয় না, মেমরি তে লোড বা সংরক্ষণ করা হয়। সুতরাং, ভার্চুয়াল মেমরি ধারণাটি কী তা হল ডিম্যান্ড পেজিং এক্সিকিউশনের সময় এই এলাকার আধিকারিকদের সুবিধা গ্রহণ করে।

(স্লাইডটাইম পড়ুন: 11:43)

একটি ডেম্যান্ড পেজ পরিকল্পনাতে, কি ঘটবে তা হল আমরা একটি হার্ড ডিস্কের মত একটি দ্বিতীয় স্টোরেজ ডিভাইসে থাকি, একটি বিশেষ স্থান যা সোয়াপ স্পেস হিসাবে বরাদ্দ করা হয়। সুতরাং, এই সোয়াপ স্থান এ এক্সিকিউটেবলের সব ব্লক উপস্থিত হবে। সুতরাং, উদাহরণস্বরূপ যদি প্রসেসটির ছয়টি ব্লক থাকে, 1 থেকে 6 পর্যন্ত তবে 6 ব্লকই সোয়াপ স্পেসে উপস্থিত হবে। এবং শুধুমাত্র চাহিদার উপর, ব্লকগুলি অটোমেটিক সোয়াপ স্পেস থেকে RAM এ লোড হবে। সুতরাং, এখন আমরা অনুমান করি যে একটি ইন্সট্রাকসন যা পেজ ফ্রেম 8 এ এক্সিকিউট করছে যা প্রসেসকে ব্লক 6 এর সাথে তুলনা করে। এবং, এর ফলস্বরূপ ব্লক 5 এর একটি বিশেষ মেমরি অবস্থানে লোড বা সংরক্ষিত হয়েছে। তাই, প্রসেসরটি ব্লক নম্বর 5 এর সাথে সংশ্লিষ্ট প্রসেস পেজ টেবিলের সন্ধান করবে এবং দেখবে যে বর্তমান বিটটি 0 তে রয়েছে। তাই, এর ফলে পেজ ফল্ট ট্রাপ হতে পারে, যা পেজ ফল্ট ইন্টারাপ্ট হিসাবেও পরিচিত। এখন, পেজ ফল্ট ইন্টারাপ্টটি তারপর অপারেটিং সিস্টেমকে সক্রিয় করবে এক্সিকিউট করার জন্য। এখন, অপারেটিং সিস্টেম তারপর সনাক্ত করবে যে এটি একটি পেজ ফল্ট ইন্টারাপ্ট ছিল। এবং, সেটা এই পেজ ফল্ট ইন্টারাপ্টের কারণটি নির্ধারণ করবে এবং এটি ডিস্ক থেকে RAM এ সংশ্লিষ্ট পেজটি লোড করবে। সেইসঙ্গে, এটি প্রসেসের পেজ টেবিলেও পরিবর্তন করবে। উদাহরণস্বরূপ, এখানে 1 এর একটি মান পেজ ফ্রেম নাম্বারে যুক্ত করা হয় এবং বর্তমান বিটটি 1 এ সেট করা হয়। এখন, এই নির্দিষ্ট ব্লক 5 এর পরবর্তীতে অ্যাক্সেসের ফলে কোন পেজ ফল্ট হতে পারে না কারণ বর্তমান বিট 1 এ সেট করা আছে। যাইহোক, 2, 3 এবং 4 ব্লক প্রথম অ্যাক্সেস করার ফলে পেজ ফল্ট হয়। এবং, এটি অপারেটিং সিস্টেমটি এক্সিকিউট করার জন্য এবং সংশ্লিষ্ট ব্লকগুলিকে RAM-র মধ্যে লোড করবে। ফলস্বরূপ, সেই নির্দিষ্ট প্রক্রিয়ার জন্য পেজ টেবিলের আপডেট করা হবে।

(স্লাইডটাইপ পড়ুন: 14:10)

তাই এখন আমরা এমন একটি অবস্থানে পৌঁছেছি যেখানে প্রতিটি পেজ ফ্রেমটি কিছু ব্লক জুড়ে রয়েছে। সুতরাং, এই ব্লকটি প্রসেস 1 বা প্রসেস 2 বা প্রসেস 3 থেকে হতে পারে। এখন, যদি প্রসেস 1 এর তৃতীয় ব্লকের মেমোরি অ্যাক্সেস করতে হয় তবে কি হবে? সুতরাং, যেহেতু আমরা দেখতে পাচ্ছি যে বর্তমান বিট প্রসেস পেজ টেবিলে 0 তে সেট করা আছে, এর ফলে একটি পেজ ফল্ট ইন্টারাপ্ট হবে। এবং, এটি অপারেটিং সিস্টেমকে এক্সিকিউট করতে ডিগার করবে। এখন, অপারেটিং সিস্টেম কি করবে? সুতরাং, তৃতীয় ব্লকের এই বিশেষ মেমরি অ্যাক্সেসটি পূরণ করার

জন্য, অপারেটিং সিস্টেমটি এই পেজগুলির মধ্যে একটিকে সরিয়ে ফেলতে হবে। মূলত, নতুন ব্লকের পথ তৈরি করার জন্য এই পেজের একটি ফ্রেম সরিয়ে ফেলা প্রয়োজন। সুতরাং, অপ্রত্যাশিত পদ্ধতিতে যে সিদ্ধান্ত নেওয়া হবে তা হল এই ব্লকটি কে অপসারণ করা উচিত, যাতে নতুন ব্লকের পথ তৈরি করতে হয়। সুতরাং, এই পছন্দটি অপারেটিং সিস্টেমে উপস্থিত কিছু অ্যালগরিদম উপর ভিত্তি করে। সুতরাং, এটি পেজ রিপ্লেসমেন্ট অ্যালগরিদম (page replacement algorithm) হিসাবে পরিচিত হয়। এবং, এর জন্য বেশ কয়েকটি পছন্দ আছে; যেমন ফার্স্ট ইন ফার্স্ট আউট, লীস্ট রিসেন্টলি ইউজড (least recently used) বা লীস্ট ফ্রিকুয়েন্টলি ইউজড (least frequently used)। সুতরাং, পেজ রিপ্লেসমেন্ট অ্যালগরিদমের সিদ্ধান্তের উপর ভিত্তি করে, অপারেটিং সিস্টেম র‍্যাম থেকে একটি ব্লক অদলবদল করা হবে এবং সম্প্রতি অ্যাক্সেস করা হয়েছে যে তৃতীয় ব্লকের সঙ্গে এটি প্রতিস্থাপন করবে। ফলস্বরূপ, ব্লকটির সাথে সামঞ্জস্যপূর্ণভাবে বর্তমান বিটটি 0 তে নির্ধারণ করা হবে। সুতরাং, আমরা র‍্যাম থেকে ব্লক নম্বর 1 কে সরিয়ে দিয়েছিলাম। এবং অতএব, ব্লক এক এর সাথে সংশ্লিষ্ট বর্তমান বিট 0 তে সেট করা হবে। এখন, 3 ব্লক যা নির্দিষ্ট পেজ ফ্রেমের মধ্যে লোড হয়েছে তার পেজ ফ্রেম নম্বর সেট থাকবে এবং বর্তমান বিটটি 1 এ সেট হবে। সুতরাং, এই প্রক্রিয়াটি র‍্যাম থেকে একটি ব্লক মুছে ফেলার এবং এটি অন্যের দ্বারা প্রতিস্থাপন করা সোয়াপ ইন এবং সোয়াপ আউট হিসাবে পরিচিত হয়, যথাক্রমে। র‍্যাম থেকে ডিস্ক পর্যন্ত একটি ব্লক সরাতে যাওয়ার প্রক্রিয়া একটি সোয়াপ আউট প্রসেস হিসাবে পরিচিত; যখন, ডিস্ক থেকে ব্লকটিকে র‍্যামে লোড করার প্রক্রিয়াটি সোয়াপ ইন হিসাবে পরিচিত হয়। এখন, সোয়াপ ইন প্রক্রিয়াটি চলাকালে, হার্ডডিস্কে ব্লক 1 সংরক্ষণ করা হচ্ছে, প্রসেসটি চলাকালে যে সমস্ত পরিবর্তন ঘটেছিল, ব্লক 1 এ যে সমস্ত পরিবর্তন ঘটে সেগুলি সোয়াপ স্থানটিতে আপডেট হবে। এইভাবে, যে ব্লকটি অবশেষে সোয়াপ স্পেসে সংরক্ষণ করা হয় সেটি ডাটাবারের সর্বশেষ ভিউ হবে। তাই, পরের জিনিসটি আমরা জিজ্ঞাসা করতে যাচ্ছি যে যদি সোয়াপ আউট হয়, অর্থাৎ, ব্লক 1 এর অনুলিপিটি RAM থেকে ডিস্ক পর্যন্ত কপি করা কী সত্যিই দরকার?

(স্লাইডটাইম পড়ুন: 18:12)

আসুন আমরা মনে করি যে ব্লক 1 এখানে 14 তম ফ্রেমে উপস্থিত ছিলো যাতে এমন কোনো তথ্য নেই যা পরিবর্তিত হয়। উদাহরণস্বরূপ, এটি ইন্সট্রাকশন অনুরূপ বা শুধুমাত্র তথ্য পড়তে পারে। এবং সেইজন্য, এই তথ্য আসলে কোনও পরিবর্তন হয় না। এই ক্ষেত্রে, সোয়াপ আউট প্রক্রিয়ায় ডিস্ক এ এই পেজ ফ্রেমের বিষয়বস্তু আবার কপি করার প্রয়োজন হবে না। অন্যদিকে, ধরুন আমাদের ব্লক নম্বর 1 তে তথ্য আছে যা সংশোধন করা হয়েছে, তাই এটি ইঙ্গিত করে যে, RAM এর মধ্যে ব্লক 1 টির একটি কপি ডিস্ক থেকে ভিন্ন। এবং তাই, সমগ্র পেজ ফ্রেমটি সোয়াপ আউট করা উচিত এবং আবার ডিস্কের মধ্যে ফিরে আসবে। সুতরাং, কিভাবে একটি অপারেটিং সিস্টেম জানবে সোয়াপ আউট প্রয়োজন কিনা? সুতরাং, এটি পেজ টেবিলের মধ্যে বিদ্যমান একটি অতিরিক্ত বিট দ্বারা সম্পন্ন হয়, যা ডি-বিট বা ডার্টি বিট (dirty bit) হিসাবে পরিচিত। সুতরাং, ডার্টি বিট মূলত ইঙ্গিত করে যে,

র‍্যামের ব্লকের বিষয়বস্তু ডিস্ক বা সোয়্যাপ স্পেসের বিষয়বস্তুগুলির সাথে সংশোধন করা হয়েছে। সুতরাং, যদি ডার্টি বিট 1 এ সেট করা হয়, এটি ইঙ্গিত দেয় যে প্রকৃতপক্ষে সোয়্যাপ স্পেসের বিষয়বস্তুগুলির সাথে সংশ্লিষ্ট RAM-র বিষয়বস্তুগুলির মধ্যে একটি পার্থক্য রয়েছে এবং সেইজন্য একটি পেজ ফ্ল্টের সময়, পেজ ফ্রেমের বিষয়বস্তু RAM ফেরত আসবে। যদি ডার্টি বিটে 0 এর একটি মান আছে, তবে পুরো সোয়্যাপ আউট প্রক্রিয়াটি প্রয়োজন হবে না। এটি কেবল নতুন প্রসেসে অদলবদল করতে হবে এবং সেই সংশ্লিষ্ট পেজ ফ্রেমের সাথে সংশ্লিষ্ট ব্লকের প্রতিস্থাপন করতে হবে।

(স্লাইডসময় পড়ুন: 20:19)

পি এবং ডি বিট, অর্থাৎ প্রেজেন্ট এবং ডার্টি বিট, ছাড়াও একটি প্রসেসের জন্য পেজ টেবিলের সুরক্ষা বিট হিসাবে পরিচিত কিছু অতিরিক্ত বিট আছে। সুতরাং, এই সুরক্ষা বিট একটি নির্দিষ্ট ব্লকের জন্য বিভিন্ন গুণাবলী নির্ধারণ করবে। সুতরাং উদাহরণস্বরূপ, অপারেটিং সিস্টেমের প্রসেসটি বিভিন্ন ব্লকগুলিকে এক্সিকিউটেবল হিসাবে সেট করতে পারে; যার মানে হল যে ব্লকের উপস্থিত তথ্য এক্সিকিউটেবল কোডের সাথে মিলিত। এবং সেইজন্য, CPU তারপর এটি এক্সিকিউট করতে পারে। ব্লকটি কেবলমাত্র পড়ার জন্য কিনা তা অন্যান্য বিট বলতে পারে এবং সেইজন্য, সেই ব্লকের যেকোনো তথ্য পরিবর্তন করার ফলে একটি ত্রুটি বা ত্রুটি দেখা দিতে পারে। এবং এখানে বিটগুলিও নির্ধারণ করবে যে একটি নির্দিষ্ট ব্লক একটি অপারেটিং সিস্টেম কোডের সাথে সম্পর্কিত কিনা বা এটি একটি নিয়মিত ব্যবহারকারী কোড। সুতরাং, এই অতিরিক্ত তথ্য সুরক্ষা বিট দ্বারা উপস্থিত হয়। ধন্যবাদ।